

Intelligent Document Processing Service

Technical Design Report · Bluno Internship Assignment

1. Overview

This report describes the design and implementation of an end-to-end IDP service that extracts structured data from invoice and transport documents (Sea Waybills, Air Waybills). The service exposes a FastAPI REST API backed by a LangGraph pipeline that combines two independent document parsers, rule-based field extraction, and a Groq LLM fallback into a single auditable processing graph.

2. Pipeline Architecture

The pipeline is implemented as a LangGraph **StateGraph** with six nodes and two conditional routing edges. State flows through a single typed **IDPState TypedDict** — no global variables, no side channels between nodes.

STEP	NODE	DESCRIPTION
1	parse_node	docling (local, layout-aware) + LlamaParse (cloud OCR) run in parallel Outputs merged into combined text
2	classify_node	Keyword scoring across combined text LLM fallback (Groq) if score < 1 hit
3a	extract_invoice_node	Rule-based regex per field → batch LLM for low-confidence fields Line item extraction (table parsing + LLM)
3b	extract_transport_node	Same hybrid strategy for seaway/airway fields
4	validate_node	Pydantic v2 coercion, per-field Decimal, HITL flagging Per-node timing recorded
ERR	error_node	Fatal parse failure → typed ErrorResponse (never raw 500)

3. Key Design Decisions

3.1 Dual Parser with Output Merging

Docling (local, layout-aware) and LlamaParse (cloud OCR) run **in parallel**, not sequentially. Their outputs are concatenated with a separator before any extraction. This means if docling cannot recover a vendor name embedded in a logo image, LlamaParse's OCR pass may capture it as text — and vice versa for complex table structures. The cost of running both parsers on every document is justified by the significant improvement in recall on mixed-layout documents.

3.2 Hybrid Extraction — Rule-based First, LLM Fallback

Every field runs a deterministic regex extractor first. Fields whose confidence score falls below **0.7** are collected and sent to Groq in a **single batched call** — not one call per field. This minimises LLM cost and latency while handling layouts that regex cannot reliably parse. The LLM is instructed via a strict system prompt with explicit rules: ISO date format, plain numeric amounts, no hallucination.

3.3 Decimal not Float for Monetary Values

All monetary amounts use Python's **Decimal** type throughout the pipeline. IEEE 754 floating-point cannot represent $0.1 + 0.2$ exactly — a correctness issue that is unacceptable in financial document processing. Pydantic v2 serialises Decimal to string in JSON via `model_dump(mode='json')`.

3.4 Partial Success Principle

A single field extraction failure produces an **ExtractionError** entry in the errors list — it does not reject the entire response. Downstream systems receive the best available data alongside a clear machine-readable description of what failed and why. The errors field is never null — always an empty list or populated list.

4. Per-field Confidence Scoring

Every extracted field carries a numeric confidence score. Consumers can threshold on confidence to decide how much to trust each value without inspecting `extraction_method` strings.

Score	Meaning	Action
1.0	Clean regex match	Use directly
0.85	LLM extracted	Use — may need spot-check
0.7	Heuristic pattern	Use — lower trust
0.0	Not found	Sent to LLM fallback; if still missing → <code>requires_review</code>

5. Field Coverage — Invoice Documents

Field	Rule-based	LLM Fallback	Notes
<code>invoice_number</code>	Yes (1.0)	Yes	INV-#, Bill No. patterns
<code>date</code>	Yes (1.0)	Yes	Flat-text normalisation for multiline layouts
<code>due_date</code>	Label-only (1.0)	Yes	Only extracts when explicit 'Due Date' label present
<code>issuer</code>	Heuristic (0.7)	Yes	Pre-Bill-To slice; image logos → null (honest failure)
<code>recipient</code>	No	Yes (0.85)	Always sent to LLM; no reliable rule pattern
<code>total</code>	Yes (1.0)	Yes	Balance Due / Total / Amount Due patterns
<code>subtotal</code>	Yes (1.0)	Yes	Subtotal / Sub Total patterns
<code>tax</code>	Yes (1.0)	Yes	Tax / VAT / Shipping patterns
<code>currency</code>	Yes (1.0)	Yes	Symbol mapped to ISO: \$ → USD, € → EUR
<code>line_items</code>	Table parse	Yes	Markdown table rows + plain-text column alignment
<code>reference_ids</code>	Yes (1.0)	No	Must contain digit — filters partial words like 'OICE'

6. Human-in-the-Loop Flagging

After validation, the pipeline checks per-field confidence on **key fields** (`issuer`, `total`, `date`, `invoice_number` for invoices; `shipper`, `consignee`, `ports` for transport). If any key field has confidence below **0.7**, or if any `ExtractionError` was recorded, the response is flagged with `requires_review: true` and a list of `review_reasons` explaining which fields need attention. This enables a human review queue to be built on top of the API with no additional logic.

7. Known Limitations & Failure Cases

- **Vendor name in image logo:** When the issuer name appears only as a raster/vector logo at the top of the invoice (e.g. SuperStore sample), neither docling nor LlamaParse recovers it as text. The LLM cannot see images — it extracts the most prominent text name instead, which is often the customer name. Mitigation: a vision LLM (GPT-4o, LLaVA) as an additional parse step could resolve this.
- **Multi-line table layouts:** Docling sometimes places label and value on separate lines in its markdown output. Fixed by normalising all whitespace to single spaces (flat text) before running regex — this collapsed multiline `'Date:\nMar 06 2012'` into `'Date: Mar 06 2012'`.

- **False positive reference IDs:** The pattern '(?:invoice|inv)\s+([A-Z0-9]+)' can match partial words when 'INVOICE' is split across lines as 'INV\nOICE'. Fixed by requiring at least one digit in every extracted reference ID.
- **Line items in free-form text:** Some invoices describe line items in prose rather than a table. Rule-based extraction handles markdown tables and space-aligned columns; the LLM handles prose. Fully unstructured layouts may still produce empty line_items.
- **LLM hallucination risk:** The Groq fallback is instructed not to hallucinate, but llama-3.1-8b-instant occasionally infers a due_date equal to the invoice date when none exists. Mitigated by explicit system prompt instruction and a confidence score of 0.85 (not 1.0) on all LLM results.

8. Observability — Per-node Timing

Every pipeline node records its wall-clock time in milliseconds. The response includes processing_time_ms (per node) and total_time_ms. Typical breakdown on a standard invoice PDF:

Node	Typical Time	Bottleneck Reason
parse	8,000 – 12,000 ms	Docling model load + LlamaParse cloud API round-trip
classify	1 – 5 ms	Pure keyword regex — no I/O
extract	500 – 2,000 ms	Groq API call for LLM fallback fields
validate	2 – 8 ms	Pydantic coercion — pure CPU

9. LLM Choice Justification — Groq + llama-3.1-8b-instant

Groq was chosen over OpenAI or Anthropic for three reasons: (1) **Free tier** with no credit card required — appropriate for an internship assignment; (2) **Sub-second inference** — Groq's LPU hardware makes llama-3.1-8b-instant one of the fastest inference endpoints available, keeping extraction latency under 2 seconds even for batch fallback calls; (3) **JSON mode** — response_format={type: json_object} guarantees parseable output without brittle string manipulation. llama-3.1-8b-instant is sufficient for structured extraction tasks — larger models (70B, GPT-4) would improve accuracy on complex layouts but at significantly higher latency and cost.